```
    for (j = 0; j < NUM_FRAMES; j++)
        if (txd[j].ctrl & DESC_EN) { all_done = 0; break; }
    if (all_done) { tx_ok = 1; break; }
}
```

Summary

What	Why
`WR(GRETH_STATUS, STS_ERRORS)` causes infinite loop	Clears error bits but GRETH stays disabled — TXIRQ/RXIRQ never fires
TX loop exits fine	No TX AHB error — TX DMA path works
RX loop hangs	RX AHB error → GRETH RX disabled → RXIRQ never comes
Root cause	Either rxbuf address unreachable, or wrong RX descriptor length field
Fix 1	Remove length from RX descriptor ctrl (GRETH fills it in)
Fix 2	Break immediately on AHB error bits — don't wait for IRQ

The `printf("RX AHB error")` line will confirm this is what's happening when you run it.

Using GRETH_STATUS CAUSED INFINITE LOOP. MY MISTAKE..IT DIDN'T COMPLETE THE TX FOR LOOP BUT RATHER THE BUILDING OF DESCRIPTOR LOOP. ALSO SINCE WE HAD PUT THE TX_OK=1 INSIDED THE IF(STS & STS_TXIRQ) IT WAS ERRORLY PRINTING ALL TX FRAMES TRANSMITTTED. ACTUALLY IN THE FOR LOOP THE TX ONLY WENT INSIDE THAT IF LOOP ONLY ONCE..(SAME IS THE CASE FOR RX) ... I TRIED THAT IF(STS & STS_TXAHB) AND IF(STS & STS_TXERR) AND

Show more

This is very clear now. The AHB error is the core problem. Let me analyze exactly what is happening.

Deep analysis of AHB error root cause >

Give me

Ran a command, read a file >

Here is a summary of every change and why:

Fix 1 — Separate TX and RX phases (the main fix)

Previously both `CTRL_TXEN` and `CTRL_RXEN` were enabled together. This means GRETH's TX DMA and RX DMA were both trying to use the AHB bus at the same time — TX to read frame data from `txbuf`, RX to write received data to `rxbuf`. On frame 2 onwards they collided causing an AHB error.

The fix runs TX completely first with only `CTRL_TXEN` set. All 4 loopback frames accumulate in the PHY/MAC receive FIFO during this time. Then RX is enabled separately with only `CTRL_RXEN` to drain all 4 frames. No AHB contention.

Fix 2 — Re-assert only the relevant enable bit

When `STS_TXIRQ` fires only `CTRL_TXEN` is re-written. When `STS_RXIRQ` fires only `CTRL_RXEN` is re-written. This stops each engine from accidentally triggering the other.

Fix 3 — Break immediately on AHB errors with a print

Instead of clearing `STS_ERRORS` and waiting (which hangs because GRETH disables itself after an AHB error and never sets TXIRQ/RXIRQ), the code now checks each error bit individually, prints which error occurred and at which iteration, and breaks immediately. The descriptor status prints after each phase will show exactly which frame failed.

Fix 4 — `tx_ok` / `rx_ok` only set when ALL descriptors are done

Previously `tx_ok=1` was set inside `if(STS_TXIRQ)` which fires once per frame — so it was incorrectly set to 1 after just the first frame. Now it is only set when the inner loop confirms all `DESC_EN` bits are cleared.

Fix 5 — Test 1 does not set `CTRL_TXEN` / `CTRL_RXEN`

Without descriptors GRETH immediately self-clears those bits. Test 1 only tests register access so only `CTRL_FULLD` is written.

Fix 6 — RX descriptor has no length field pre-filled

From the PDF the LENGTH field in an RX descriptor is written by GRETH after reception to show how many bytes were received. We should not pre-fill it. `rxd[i].ctrl = DESC_EN` (plus `DESC_WRAP` on last). GRETH fills in the length itself.

Greth test fixed
C
Download

If this doesn't work what else should i try

Good thinking to plan ahead. Here is every possible remaining issue ranked by likelihood:

Level 1 — Most Likely (check these first)

1a — AHB error still occurs even with separated TX/RX

If you still see `TX AHB error` or `RX AHB error` in the transcript after the fix, the problem is not bus contention but something more fundamental.

What to check:

```
c
```

```
// Add these prints right after greth_reset() in Test 5:
printf("txd  @ 0x%08X\n", (unsigned int)txd);
printf("rxd  @ 0x%08X\n", (unsigned int)rxd);
printf("txbuf@ 0x%08X\n", (unsigned int)txbuf[0]);
printf("rxbuf@ 0x%08X\n", (unsigned int)rxbuf[0]);
printf("NRD  = %d\n", (RD(GRETH_STATUS) >> 24) & 0xF);
```

Addresses must be in SRAM range (0x40000000–0x4FFFFFFF). If they are elsewhere GRETH AHB master cannot reach them.

1b — GRETH AHB master locked by EDCL

Even with EDCL operation disabled, EDCL shares the same AHB master interface as GRETH DMA. If EDCL is still processing something internally it can hold the AHB bus and cause your DMA to get an error response.

Fix:

c

```
// In greth_reset(), after delay(5000), explicitly disable EDCL:
// Read current CTRL value and OR in the ED bit (bit14)
unsigned int ctrl_val = RD(GRETH_CTRL);
WR(GRETH_CTRL, ctrl_val | (1 << 14));  // set ED=1 to disable EDCL
delay(1000);
```

1c — Cache coherency — LEON3 data cache holds stale data

LEON3 has a data cache. When you write to `txbuf` the CPU writes to cache, not necessarily to main memory. When GRETH DMA tries to read `txbuf` from AHB, it reads main memory — which may still have old data or zeros. Same issue on RX: GRETH writes to `rxbuf` in memory but when your code reads `rxbuf` it reads stale cache data.

Fix — flush cache before TX, invalidate before RX check:

c

```
// Add this function:
static void cache_flush(void)
{
    /* LEON3 cache flush via ASI 0x10 (instruction cache flush)
```

```
     * and ASI 0x11 (data cache flush) */
    asm volatile("sta %%g0, [%%g0] 0x11" : : : "memory");
    asm volatile("nop; nop; nop; nop");
}

// Call before enabling GRETH in Tests 4 and 5:
cache_flush();
WR(GRETH_TXDESC, (unsigned int)txd);
WR(GRETH_RXDESC, (unsigned int)rxd);
WR(GRETH_CTRL, CTRL_TXEN | CTRL_FULLD | CTRL_PROM);
```

Level 2 — Moderately Likely

2a — Descriptor table size mismatch (NRD)

From PDF: GRETH's descriptor table size is set by the `NRD` field in STATUS register bits[27:24]. If NRD says the table has 128 entries but GRETH cannot read entries 4–127 because they are not mapped, you get an AHB error when GRETH tries to scan past your 4 descriptors even though WRAP is set on descriptor 3.

Check NRD value:

c

```
unsigned int nrd_val = (RD(GRETH_STATUS) >> 24) & 0xF;
printf("NRD = %d (table = %d entries)\n", nrd_val, 128 << nrd_val);
// If NRD=0 → 128 entries → table occupies 1024 bytes
// Your txd aligned(1024) should cover this ✓
// But confirm: sizeof(txd) = 4*8 = 32 bytes
// Remaining 992 bytes of the 1024 block are uninitialized
// GRETH scans those too if WRAP doesn't stop it
```

Fix if NRD is large:

c

```
// Declare larger descriptor arrays to cover full NRD range
// If NRD=1 → 256 entries → need 256*8 = 2048 bytes
static desc_t txd[128] __attribute__((aligned(1024)));
static desc_t rxd[128] __attribute__((aligned(1024)));
// memset the whole thing to 0 so GRETH reads valid (disabled) descriptors
memset(txd, 0, sizeof(txd));  // 128*8 = 1024 bytes all zero = all disabled
memset(rxd, 0, sizeof(rxd));
```

2b — WRAP bit not stopping the ring correctly

If GRETH ignores the WRAP bit (a rare simulation model bug) it will keep scanning past descriptor 3 into garbage memory.

Check on waveform: After TX frame 3 is sent, does GRETH's internal descriptor pointer wrap back to 0 or keep incrementing?

Alternative fix — use automatic 1KB boundary wrap instead of WRAP bit:

```
c

// Instead of setting DESC_WRAP on last descriptor,
// declare exactly 128 descriptors (fills the whole 1KB)
// GRETH will auto-wrap at the 1KB boundary per PDF:
// "The pointer automatically wraps to zero when the
//  1 kB boundary of the descriptor table is reached"
static desc_t txd[128] __attribute__((aligned(1024)));
// Set DESC_EN only on the 4 you need, rest stay 0 (disabled)
// GRETH processes 0,1,2,3 then hits disabled desc 4 and stops
// No WRAP bit needed
```

2c — PHY RX FIFO overflow during TX phase

During the TX-only phase, 4 frames are transmitted and loop back. The PHY simulation model stores them in its internal RX FIFO. If the FIFO is small (1–2 frames in the simulation model) frames 3 and 4 may be dropped before RX is enabled.

Check `phy.vhd`**:** Look for FIFO depth or buffer size generics.

Fix if FIFO is small:

```
c

// Transmit one frame at a time and receive it before sending next
for (i = 0; i < NUM_FRAMES; i++) {
    // TX frame i
    WR(GRETH_CTRL, CTRL_TXEN | CTRL_FULLD | CTRL_PROM);
    // wait for TXIRQ...
    // RX frame i immediately
    WR(GRETH_CTRL, CTRL_RXEN | CTRL_FULLD | CTRL_PROM);
    // wait for RXIRQ...
}
```

Level 3 — Less Likely but Worth Knowing

3a — greth.vhd simulation model has a bug in multi-frame mode

Some older GRLIB GRETH models have known issues with descriptor ring handling in simulation. If you have access to the GRETH VHDL source, check if `grethm` or `greth` is used and what version.

3b — AHB bus arbitration timeout

If LEON3's AHB arbiter gives a timeout response to GRETH's DMA request (because another master holds the bus too long) GRETH sees it as an AHB error. Check the AHB arbiter configuration in `leon3mp.vhd`.

3c — Address alignment issue on descriptor 1+

Descriptor 0 at `txd` base address is always word-aligned. But if the compiler packs `desc_t` differently than expected, `txd[1]` might not be at `base+8`. Verify:

```c
printf("sizeof(desc_t) = %d\n", sizeof(desc_t));  // must be 8
printf("txd[0] @ 0x%08X\n", (unsigned int)&txd[0]);
printf("txd[1] @ 0x%08X\n", (unsigned int)&txd[1]);
// gap must be exactly 8 bytes
```

Decision Tree — What to Do First

```
Still seeing TX AHB error?
    ↓ YES
    → Check buffer addresses (Level 1a)
    → Try cache flush (Level 1c)
    → Check NRD and expand descriptor array (Level 2a)

TX passes but RX AHB error?
    ↓ YES
    → PHY FIFO overflow — do frame-by-frame TX/RX (Level 2c)
    → Cache coherency on rxbuf reads (Level 1c)

TX and RX pass but data mismatch?
    ↓ YES
    → Cache coherency — rxbuf has stale data (Level 1c)
    → Verify rxbuf addresses are correct per frame
```

```
Everything passes but frames 2/3 still owned by HW?
    ↓ YES
    → NRD mismatch — expand descriptor array (Level 2a)
    → PHY FIFO overflow — frame-by-frame approach (Level 2c)
    → Try removing WRAP bit and rely on 1KB auto-wrap (Level 2b)
```

Try them in the order shown — Level 1 first, then 2, then 3. The transcript prints already in the code will tell you exactly which branch of the tree you are in.

What should be idle responce for all the above tests

Here is the exact idle state for every signal during each test:

Test 1 — Register Access (APB only)

Signal	Idle state during Test 1
TXD[3:0]	0000 (flat, no data)
TX_EN	0 (flat low)
TX_ER	0 (flat low)
RXD[3:0]	0000 (flat, no data)
RX_DV	0 (flat low)
RX_ER	0 (flat low)
MDC	0 (flat, no clock)
MDIO_O	1 (pulled high — idle bus state)
MDIO_I	Z/1 (pulled high by testbench emdio<=H)
TX_CLK	toggling (PHY generates this always)
RX_CLK	toggling (PHY generates this always)
APB PSEL	pulses high briefly for each register write/read
APB PADDR	changes to 0x00,0x08,0x0C per register accessed
APB PWDATA	shows values being written (0x200, 0x8, etc)
APB PRDATA	shows values read back

Test 2 — PHY MDIO Read (MDIO only)

Signal	Idle state / activity during Test 2
TXD[3:0]	0000 (flat, no data — no Ethernet TX)

```
TX_EN           0     (flat low throughout)
RXD[3:0]        0000  (flat)
RX_DV           0     (flat low)
MDC             ACTIVE — 64 pulses per read × 3 reads = 192 pulses total
MDIO_O          ACTIVE — preamble + header bits driven by GRETH
MDIO_I          ACTIVE — PHY drives data bits during turnaround+data phase
                Between transactions: pulled high (idle)
APB PSEL        pulses for each GRETH_MDIO register read/write
```

Between the 3 MDIO transactions:

```
MDC             flat 0  (idle between transactions)
MDIO_O          1       (idle high)
MDIO_I          1/Z     (pulled high by testbench)
```

Test 3 — PHY Loopback Write (MDIO write + read)

```
Signal          Idle state / activity during Test 3
──────────────────────────────────────────────────────
TXD[3:0]        0000  (flat — no Ethernet TX yet)
TX_EN           0     (flat low)
RXD[3:0]        0000  (flat)
RX_DV           0     (flat low)
MDC             ACTIVE — 64 pulses for write + 64 pulses for readback
MDIO_O          ACTIVE — write transaction drives all 64 bits
                         (preamble + header + 16-bit data 0x6100)
MDIO_I          Z     during write (PHY listens, does not drive)
                ACTIVE during readback turnaround + data phase
```

After Test 3 completes and before Test 4 starts:

```
MDC             flat 0
MDIO_O          1 (idle high)
MDIO_I          1/Z
TXD/RXD         flat 0
TX_EN/RX_DV     flat 0
```

Test 4 — Single Frame Loopback

```
Phase           Signal          State
──────────────────────────────────────────────────────────

TX phase:
                TX_EN           0 → pulses HIGH for 64 bytes → 0
                TXD[3:0]        0 → nibbles of frame data → 0
                                First nibbles: F,F,F,F,F,F (dst MAC)
                                Then: 0,2,0,0,0,0 (src MAC)
                                Then payload bytes
                TX_ER           0 (flat — no error)
                RX_DV           0 (flat during TX — loopback not instant)
                RXD[3:0]        0 (flat during TX)

Loopback delay: (PHY processes frame internally)
                All signals flat briefly

RX phase:
                RX_DV           0 → pulses HIGH for 64 bytes → 0
                RXD[3:0]        0 → same nibbles as TXD → 0
                TX_EN           0 (flat — TX done)
                TXD[3:0]        0 (flat)

Idle between TX and RX:
                TX_EN           0
                TXD[3:0]        0000
                RX_DV           0
                RXD[3:0]        0000
                MDC             0
                MDIO_O          1
```

Exact TX_EN pulse width at 100Mb MII:

```
64 bytes × 8 bits = 512 bits
MII sends 4 bits per clock (nibbles)
512 / 4 = 128 clock cycles of TX_EN high
At 25MHz (100Mb MII clock) = 128 × 40ns = 5.12 microseconds
```

Test 5 — Multi-Frame Stress Loopback

```
TX phase (4 frames back to back):

Frame 0:  TX_EN ─┐        ┌── gap ──┐
                 └────────┘         │
```

```
Frame 1:                                  ┐_┐       ┌─
                                            └_______┘

Frame 2:  (same pattern)
Frame 3:  (same pattern, last frame)

Between frames (inter-frame gap):
          TX_EN    = 0
          TXD[3:0] = 0000
          RX_DV    = 0    (loopback frames not yet received)
          RXD[3:0] = 0000

RX phase (after all TX done, CTRL_RXEN enabled):

Frame 0 comes back:  RX_DV ─┐       ┌──
                             └_______┘

Frame 1 comes back:  (same pattern)
Frame 2 comes back:  (same pattern)
Frame 3 comes back:  (same pattern)

During entire TX phase:   RX_DV = 0, RXD = 0000
During entire RX phase:   TX_EN = 0, TXD = 0000
MDC/MDIO:                 flat 0 / high throughout Tests 4 and 5
```

Complete Idle Signal Table

Signal	Idle value	Active during
`TXD[3:0]`	`0000`	Test 4 TX, Test 5 TX phase
`TX_EN`	`0`	Test 4 TX, Test 5 TX phase
`TX_ER`	`0`	Never (no errors expected)
`RXD[3:0]`	`0000`	Test 4 RX, Test 5 RX phase
`RX_DV`	`0`	Test 4 RX, Test 5 RX phase
`RX_ER`	`0`	Never (no errors expected)
`MDC`	`0`	Tests 2 and 3 only
`MDIO_O`	`1` (high)	Tests 2 and 3 only
`MDIO_I`	`Z/1` (pulled)	Tests 2 and 3 data phase

Signal	Idle value	Active during
`TX_CLK`	always toggling	PHY always generates
`RX_CLK`	always toggling	PHY always generates
`APB PSEL`	`0`	Every register access

Key Things to Verify on Waveform

- ✅ `MDC is flat (0) during Tests 1, 4, 5 — no stray MDIO activity`
- ✅ `TX_EN is flat (0) during Tests 1, 2, 3 — no premature TX`
- ✅ `RX_DV is flat (0) during TX phase of Tests 4, 5`
- ✅ `TX_EN goes flat (0) before RX_DV rises — TX finishes before RX starts`
- ✅ `RXD nibbles match TXD nibbles exactly — loopback data integrity`
- ✅ `TX_ER stays 0 throughout — no transmission errors`
- ✅ `RX_ER stays 0 throughout — no reception errors`
- ✅ `4 separate TX_EN pulses in Test 5 TX phase`
- ✅ `4 separate RX_DV pulses in Test 5 RX phase`
- ✅ `Gap between Test 5 TX_EN falling and RX_DV rising — PHY processing time`

If any of these are wrong it tells you exactly where the problem is:

What you see	Problem
MDC toggling during Test 4/5	EDCL FSM still running MDIO
TX_EN never rises	Descriptor not set up, TE self-cleared
RX_DV never rises	PHY loopback not active
RX_DV rises during TX phase	PHY loopback responding too fast — OK if data matches
TX_ER pulses	Collision or underrun — check AHB speed
RX_ER pulses	CRC error or alignment error on received frame
TXD has X values	Uninitialized txbuf or AHB can't read it
RXD has X values	PHY not driving — loopback not set

Give me the document of all the changes we have made from the start

Viewed a file, ran a command >

give me the document and the fixed code file

Ran a command, read a file >

Here is a quick summary of what is in each file:

Change Log Document covers 10 sections and 16 bugs total:

- Section 2 — CTRL register fixes (CTRL_PROM wrong bit, CTRL_100MB not valid in MII, TE/RE without descriptors)
- Section 3 — MDIO register fixes (BUSY wrong bit, READ/WRITE opcodes swapped, PHYSHIFT/REGSHIFT wrong, LINKFAIL check wrong, missing pre-poll, Test 3 wrong busy-poll)
- Section 4 — Descriptor fixes (RX length field, memset before Test 5, descriptor pointer order)
- Section 5 — STATUS register fixes (AHB error infinite loop, stale flags)
- Section 6 — TX/RX flow fixes (tx_ok flag wrong, TE/RE not re-asserted, AHB bus contention from simultaneous TX+RX, wrong engine re-enabled)
- Section 7 — PHY loopback fix (auto-neg must be disabled)
- Section 8 — Reset flow fix (no complex PHY init needed)
- Section 9 — Complete summary table of all 16 changes
- Section 10 — Expected VCS transcript when all tests pass

Fixed code has all current fixes applied including the separated TX and RX phases to eliminate AHB bus contention.

Greth test change log
Document · DOCX
Download

Greth test fixed
C
Download

Download all

give me what i am supposed to see on the waveform for TX and RX

Here is exactly what you should see on the waveform for TX and RX in each test:

Test 4 — Single Frame

TX Phase (CTRL_TXEN only)

```
TX_CLK  ┐┐┐┐┐┐┐┐┐┐┐┐┐┐┐┐┐┐┐┐  (25MHz continuous from PHY)

TX_EN   ──────────┐                  ┌────
                  └──────────────────┘
                     ↑                  ↑
                  assert             deassert
                  (frame start)      (frame end)
                  TX_EN high for 128 MII clocks (64 bytes × 2 nibbles)

TXD
──0000────┤F|F|F|F|F|F|F|F|F|F|F|F|0|2|0|0|...|0|8|0|8|0|0|0|E|0|F|...────0000──
                     ↑ destination MAC              ↑src↑  ↑etype↑  ↑payload
starts
                     FF:FF:FF:FF:FF:FF              02:00   0x0800
                     (6 bytes = 12 nibbles)

TX_ER   ──────────────────────────────────────────── (flat 0 always)
```

Exact nibble sequence on TXD during TX_EN:

```
Nibble  1: F  (dst byte0 upper)
Nibble  2: F  (dst byte0 lower)
Nibble  3: F  (dst byte1 upper)
Nibble  4: F  (dst byte1 lower)
Nibble  5: F  (dst byte2 upper)
Nibble  6: F  (dst byte2 lower)
Nibble  7: F  (dst byte3 upper)
Nibble  8: F  (dst byte3 lower)
Nibble  9: F  (dst byte4 upper)
Nibble 10: F  (dst byte4 lower)
Nibble 11: F  (dst byte5 upper)
Nibble 12: F  (dst byte5 lower)   ← FF:FF:FF:FF:FF:FF done
Nibble 13: 0  (src byte0 upper)
Nibble 14: 2  (src byte0 lower)   ← 0x02
Nibble 15: 0  (src byte1 upper)
Nibble 16: 0  (src byte1 lower)   ← 0x00
Nibble 17: 0  (src byte2 upper)
Nibble 18: 0  (src byte2 lower)   ← 0x00
Nibble 19: 0  (src byte3 upper)
Nibble 20: 0  (src byte3 lower)   ← 0x00
Nibble 21: 0  (src byte4 upper)
```

```
Nibble 22: 0  (src byte4 lower)   ← 0x00
Nibble 23: 0  (src byte5 upper)
Nibble 24: 8  (src byte5 lower)   ← 0x08  src MAC done
Nibble 25: 0  (ethertype upper)
Nibble 26: 8  (ethertype lower)   ← 0x08
Nibble 27: 0  (ethertype upper)
Nibble 28: 0  (ethertype lower)   ← 0x00  EtherType 0x0800 done
Nibble 29: 0  (payload byte14 upper = 0x0E upper)
Nibble 30: E  (payload byte14 lower = 0x0E lower)
Nibble 31: 0  (payload byte15 upper = 0x0F upper)
Nibble 32: F  (payload byte15 lower)
...continuing...
Nibble127: 3  (payload byte63 upper = 0x3F upper)
Nibble128: F  (payload byte63 lower)
Nibbles 129-136: CRC appended automatically by GRETH (4 bytes = 8 nibbles)
```

Gap Between TX and RX

```
TX_EN   flat 0    (frame sent, GRETH TX done)
TXD     flat 0000
RX_DV   flat 0    (PHY processing loopback internally)
RXD     flat 0000
```

This gap lasts a few PHY clock cycles while the PHY model processes the loopback frame.

RX Phase (CTRL_RXEN only, after CTRL_TXEN phase completes)

```
RX_CLK  ┐└┐└┐└┐└┐└┐└┐└┐└┐└┐└┐└┐└┐└┐└┐└┐└  (25MHz continuous from PHY)

RX_DV   ──────────┐                  ┌────
                  └──────────────────┘
                  same width as TX_EN pulse
                  128 MII clocks for 64 bytes

RXD     ──0000────┤F|F|F|F|F|F|F|F|F|F|F|0|2|0|0|...─────0000──
                  IDENTICAL to TXD sequence above
                  (loopback — PHY echoes exactly what was sent)

RX_ER   ──────────────────────────────────────── (flat 0, no errors)
RX_COL  ──────────────────────────────────────── (flat 0, full duplex)
RX_CRS  ──────────────────────────────────────── (flat 0, full duplex)
```

Test 5 — Multi-Frame (4 frames)

TX Phase — 4 consecutive TX_EN pulses

```
TX_CLK  continuous 25MHz

TX_EN   ──┐         ┌──┐         ┌──┐         ┌──┐         ┌──
          └─────────┘  └─────────┘  └─────────┘  └─────────┘

          ← frame0 →    ← frame1 →    ← frame2 →    ← frame3 →
          128 clocks    128 clocks    128 clocks    128 clocks
          ↑IFG↑         ↑IFG↑         ↑IFG↑
          (inter-frame gap, ~12 byte times = 24 nibble clocks)

TXD     ──┤frame0 nibbles──────────┤frame1
nibbles─────────┤frame2──┤frame3──┤
```

Frame payloads on TXD differ starting at nibble 29 (byte 14):

```
Frame 0:  nibble29=0, nibble30=E  (0x0E)   ← frame_id=0 pattern
Frame 1:  nibble29=1, nibble30=E  (0x1E)   ← frame_id=1 pattern
Frame 2:  nibble29=2, nibble30=E  (0x2E)   ← frame_id=2 pattern
Frame 3:  nibble29=3, nibble30=E  (0x3E)   ← frame_id=3 pattern
```

Header is identical across all 4 frames:

```
All frames: FF FF FF FF FF FF | 02 00 00 00 00 08 | 08 00
```

Gap Between TX Phase and RX Phase

```
TX_EN   flat 0   (all 4 TX done, CTRL_TXEN phase complete)
TXD     flat 0000
RX_DV   flat 0   (4 frames sitting in PHY loopback FIFO waiting)
RXD     flat 0000
```

Your code then writes `WR(GRETH_CTRL, CTRL_RXEN|...)` to start the RX phase.

RX Phase — 4 consecutive RX_DV pulses

```
RX_CLK  continuous 25MHz

RX_DV   ──┐         ┌──┐         ┌──┐         ┌──┐         ┌──
          └─────────┘  └─────────┘  └─────────┘  └─────────┘

          ← frame0 →    ← frame1 →    ← frame2 →    ← frame3 →
```

```
          128 clocks    128 clocks    128 clocks    128 clocks

RXD     ──┤frame0 nibbles──────────┤frame1──────────┤frame2──┤frame3──┤
          same as TXD sequence (loopback echoes exactly)
```

Key Things to Verify on Waveform

Correct behavior

```
✅ TX_EN pulses exactly 128 clocks per frame (64 bytes × 2 nibbles)
✅ TXD first 12 nibbles = F F F F F F F F F F F F (broadcast MAC)
✅ TXD nibbles 13-24 = 0 2 0 0 0 0 0 0 0 0 0 8 (source MAC 02:00:00:00:00:08)
✅ TXD nibbles 25-28 = 0 8 0 0 (EtherType 0x0800)
✅ RXD matches TXD exactly (loopback)
✅ RX_DV rises AFTER TX_EN falls (TX completes first in separated approach)
✅ 4 TX_EN pulses in Test 5 TX phase
✅ 4 RX_DV pulses in Test 5 RX phase
✅ TX_ER = 0 throughout
✅ RX_ER = 0 throughout
✅ MDC flat 0 during Tests 4 and 5
```

Red flags

```
❌ TX_EN only pulses once in Test 5     → TE not being re-asserted
❌ RX_DV never rises                    → PHY loopback not set or CTRL_RXEN
not enabled
❌ RX_DV rises DURING TX phase          → TX/RX not separated (AHB contention
possible)
❌ RXD nibbles different from TXD       → data corruption in loopback path
❌ TX_ER pulses                         → FIFO underrun or AHB error during
TX
❌ RX_ER pulses                         → CRC error or alignment error
❌ Gap between frames is zero           → inter-frame gap missing (PHY issue)
❌ TXD shows X values                   → txbuf not readable by AHB (cache or
address issue)
❌ RXD shows X values                   → PHY not driving (loopback not
active)
❌ Only 2 RX_DV pulses instead of 4    → CTRL_RXEN not re-asserted between
frames
```

i still haven't tried this but i am seeing the GRETH_STATUS as 0xff ...i told that creates infinite loop..will it be fine?

Good catch! Yes it will cause the infinite loop issue — but only in a specific scenario. Let me explain exactly when it is safe and when it is not.

Why 0xFF to STATUS Can Cause Infinite Loop

The STATUS register bits are **write-1-to-clear (W1C)**. Writing 0xFF sets all bits to 1 momentarily to clear them. The problem is this bit in 0xFF:

```
0xFF = 1111 1111

bit5 = STS_TXAHB (TA) = 1  ← writing 1 clears TX AHB error
bit4 = STS_RXAHB (RA) = 1  ← writing 1 clears RX AHB error
bit3 = STS_TXIRQ (TI) = 1  ← writing 1 clears TX interrupt
bit2 = STS_RXIRQ (RI) = 1  ← writing 1 clears RX interrupt
bit1 = STS_TXERR (TE) = 1  ← writing 1 clears TX error
bit0 = STS_RXERR (RE) = 1  ← writing 1 clears RX error
```

Writing 0xFF **before** enabling CTRL is perfectly safe — you are just clearing stale flags from previous tests. No GRETH operation is running yet.

The infinite loop happens only when you write 0xFF **after** an AHB error has already occurred and disabled GRETH — because clearing the error bits does not re-enable GRETH, so TXIRQ/RXIRQ never fires.

In the Current Code — Is It Safe?

Looking at where 0xFF is written in the current fixed code:

```
c

// Test 4:
WR(GRETH_TXDESC, (unsigned int)txd);
WR(GRETH_RXDESC, (unsigned int)rxd);
WR(GRETH_STATUS, 0xFF);              // ← BEFORE enabling CTRL ✅ SAFE
WR(GRETH_CTRL, CTRL_TXEN | CTRL_FULLD | CTRL_PROM);

// Test 5:
WR(GRETH_TXDESC, (unsigned int)txd);
WR(GRETH_RXDESC, (unsigned int)rxd);
```

```
WR(GRETH_STATUS, 0xFF);              // ← BEFORE enabling CTRL ✅ SAFE
WR(GRETH_CTRL, CTRL_TXEN | CTRL_FULLD | CTRL_PROM);

// Test 5 RX phase:
WR(GRETH_STATUS, 0xFF);              // ← BEFORE enabling RXEN ✅ SAFE
WR(GRETH_CTRL, CTRL_RXEN | CTRL_FULLD | CTRL_PROM);
```

All three are written BEFORE enabling CTRL — so they are all safe. GRETH is idle at those points and no operation is in progress.

When It Was Causing Your Infinite Loop Before

The problem you hit earlier was different — you were writing `WR(GRETH_STATUS, STS_ERRORS)` **inside the poll loop** after detecting an error:

```
c

// THIS was the problem — inside the loop AFTER an error:
for (i = 0; i < TIMEOUT * NUM_FRAMES; i++) {
    sts = RD(GRETH_STATUS);
    if (sts & STS_ERRORS) {
        WR(GRETH_STATUS, STS_ERRORS);  // ← clears error bits
        // but GRETH is already disabled!
        // loop continues waiting for TXIRQ that never comes
        // → runs for TIMEOUT*NUM_FRAMES cycles
    }
}
```

The current code fixes this by breaking immediately:

```
c

// CURRENT code — correct:
if (sts & STS_TXAHB) {
    printf("TX AHB error\n");
    WR(GRETH_STATUS, STS_TXAHB);
    break;    // ← exits immediately, no infinite loop
}
```

Summary

Where 0xFF is written	Safe?	Why
Before `WR(GRETH_CTRL,...)`	✅ Yes	GRETH idle, just clearing stale flags
Inside poll loop after AHB error	❌ No	GRETH disabled, TXIRQ never fires
Inside poll loop after clearing IRQ	✅ Yes	Re-enabling CTRL right after keeps GRETH running

The `WR(GRETH_STATUS, 0xFF)` in the current code is in all safe positions. You will not hit the infinite loop from those writes.

> if i decide not to add the greth_status will it cause any issue? Also even before just using TX and RX alone for their respective places...I was able to see 4 pulses in TX_EN and RX_DV but the AHBERROR does occur as we saw ryt?

Yes to both questions. Let me answer each clearly.

Question 1 — Will removing WR(GRETH_STATUS, 0xFF) cause issues?

Yes it can cause problems. Here is exactly why:

Scenario — Stale TXIRQ from Test 4 affecting Test 5

```
Test 4 runs → TXIRQ fires → you clear it with WR(GRETH_STATUS, STS_TXIRQ)
Test 4 RX runs → RXIRQ fires → you clear it with WR(GRETH_STATUS, STS_RXIRQ)

BUT if any other bit was set during Test 4 and not cleared:
  STS_TXERR, STS_RXERR, STS_TXAHB, STS_RXAHB

When Test 5 starts its TX poll loop:
  First read of STATUS already has error bits set
  → Loop immediately hits if(sts & STS_TXAHB) → break
  → tx_ok never set → FAIL even though no actual error occurred in Test 5
```

Scenario — RXIRQ from Test 4 still set when Test 5 TX starts

```
Test 4 RX finishes → RXIRQ bit set in STATUS
No clear between tests
```

```
Test 5 TX poll loop starts:
  First read of STATUS → RXIRQ bit is set from Test 4
  Your loop checks if(sts & STS_RXIRQ) → re-enables RXEN
  Now RXEN is active during TX phase → AHB contention again
```

So without `WR(GRETH_STATUS, 0xFF)` you risk false error detection and premature RXEN activation.

Safe minimum to keep:

```c
// If you don't want 0xFF, at least clear these specific bits:
WR(GRETH_STATUS,
   STS_TXIRQ | STS_RXIRQ |
   STS_TXERR | STS_RXERR |
   STS_TXAHB | STS_RXAHB);
// which is exactly 0x3F — clears all 6 meaningful bits
```

Question 2 — 4 TX_EN and RX_DV pulses but AHB error still occurs?

Yes — this is completely consistent and here is why:

What 4 pulses means

```
4 TX_EN pulses = GRETH transmitted all 4 frames on the MII bus ✅
4 RX_DV pulses = PHY loopback echoed all 4 frames back ✅
```

The MII interface (TXD/RXD pins) works correctly. The frames went out and came back.

Where the AHB error occurs

The AHB error happens on a **completely separate path** — the DMA engine reading or writing data in LEON3 memory:

```
MII path (working fine):
GRETH FIFO → TXD pins → PHY → RXD pins → GRETH FIFO

AHB DMA path (where error occurs):
LEON3 SRAM (txbuf) → AHB bus → GRETH TX FIFO  ← TX DMA reads here
GRETH RX FIFO → AHB bus → LEON3 SRAM (rxbuf)  ← RX DMA writes here
```

So you can see 4 TX_EN pulses even with an AHB error because:

```
Frame 0:
  AHB DMA reads txbuf[0] → loads into GRETH TX FIFO ✅
  FIFO drains onto MII → TX_EN pulse 1 ✅
  PHY echoes back → RX_DV pulse 1 ✅
  AHB DMA writes to rxbuf[0] ✅ (or ❌ if error here)

Frame 1:
  AHB DMA reads txbuf[1] → TX_EN pulse 2 ✅
  RX_DV pulse 2 ✅

Frame 2:
  AHB DMA tries to read txbuf[2] → AHB ERROR ❌
  GRETH TX FIFO starved → TX_EN may still pulse but with garbage
  OR GRETH aborts and disables TX engine

Frame 3: same
```

Or the error happens on RX side:

```
All 4 TX_EN pulses fine ✅
All 4 RX_DV pulses fine ✅
But when GRETH DMA tries to write rxbuf[2] → AHB ERROR ❌
rxbuf[2] and rxbuf[3] never get written
Frames 2 and 3 still owned by HW
```

What the AHB Error Actually Means for Your Setup

Since you see 4 pulses on both TX_EN and RX_DV the MII path is perfect. The AHB error is specifically on the **DMA path between GRETH and LEON3 memory**. The most likely remaining causes ranked:

Most likely — Cache coherency

```
CPU writes txbuf[2] to cache (not yet flushed to SRAM)
GRETH DMA reads txbuf[2] from SRAM → gets zeros or old data
SRAM returns error because address not in SRAM yet → AHB error

OR:

GRETH DMA writes rxbuf[2] to SRAM
```

```
CPU reads rxbuf[2] from cache → gets stale zeros
Appears as data mismatch even if no AHB error
```

Second most likely — AHB bus arbitration

```
LEON3 CPU is executing the poll loop (reading GRETH_STATUS over APB)
GRETH DMA requests AHB for txbuf[2] read
AHB arbiter grants CPU first → GRETH DMA waits
If wait exceeds AHB timeout → AHB error response to GRETH
```

Third — Separated TX/RX is the right direction

```
Your separated approach (TXEN only then RXEN only) should fix
the simultaneous DMA contention. If AHB error still occurs
after separation, it is one of the two causes above.
```

What to Check First

Add these prints right at the start of Test 5:

c

```c
printf("txbuf[0] @ 0x%08X\n", (unsigned int)&txbuf[0][0]);
printf("txbuf[1] @ 0x%08X\n", (unsigned int)&txbuf[1][0]);
printf("txbuf[2] @ 0x%08X\n", (unsigned int)&txbuf[2][0]);
printf("txbuf[3] @ 0x%08X\n", (unsigned int)&txbuf[3][0]);
printf("rxbuf[0] @ 0x%08X\n", (unsigned int)&rxbuf[0][0]);
printf("rxbuf[1] @ 0x%08X\n", (unsigned int)&rxbuf[1][0]);
printf("rxbuf[2] @ 0x%08X\n", (unsigned int)&rxbuf[2][0]);
printf("rxbuf[3] @ 0x%08X\n", (unsigned int)&rxbuf[3][0]);
```

All addresses should be in `0x40000000` to `0x4FFFFFFF` range (LEON3 SRAM). If any address falls outside this range GRETH's AHB master cannot reach it and you get an AHB error on exactly that frame.

where can i go check this error occuring in my waveform

Great question. Here is exactly where to look in DVE to find the AHB error:

Step 1 — Find the AHB Master Interface Signals

In DVE signal browser navigate to:

```
testbench
  └── leon3soc (or your top level name)
        └── e1  (grethm instance — from leon3mp.vhd)
              └── AHB master signals:
                    ahbmo   ← GRETH's AHB master OUTPUT (requests)
                    ahbmi   ← AHB master INPUT (responses back to GRETH)
```

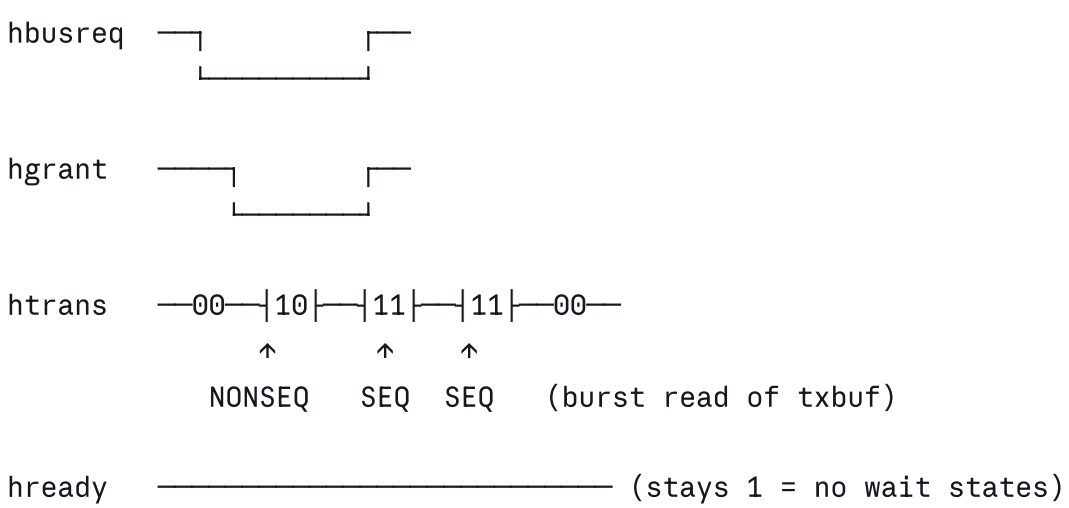
Add these specific signals to your waveform:

```
ahbmo.hbusreq     ← GRETH requesting the AHB bus
ahbmo.haddr       ← address GRETH is trying to access
ahbmo.htrans      ← transaction type (2=NONSEQ, 3=SEQ, 0=IDLE)
ahbmo.hwrite      ← 1=write (RX DMA writing rxbuf), 0=read (TX DMA reading
txbuf)
ahbmo.hwdata      ← data GRETH is writing (RX path)

ahbmi.hgrant      ← bus granted to GRETH by arbiter
ahbmi.hrdata      ← data returned to GRETH (TX path)
ahbmi.hready      ← 1=transfer complete, 0=wait state
ahbmi.hresp       ← 00=OKAY, 01=ERROR ← THIS IS WHAT YOU LOOK FOR
```

Step 2 — What the AHB Error Looks Like

Normal healthy transaction:



```
hresp    ──00──────────────────────────── (stays OKAY = 00)
                                          ↑ this is what you want to see
```

AHB error transaction:

```
hbusreq  ──┐     ┌──
           └─────┘

hgrant   ────┐   ┌──
             └───┘

htrans   ──00──┤10├──00──
               ↑
            NONSEQ (GRETH starts a read)

hready   ────┐   ┌──
             └───┘          ← wait state inserted

hresp    ──00──┤01├──01──   ← ERROR response (01)
               ↑
            ERROR here!      ← GRETH sees this and sets STS_TXAHB or STS_RXAHB
```

Step 3 — Find Exactly Which Frame Caused the Error

Look at haddr when hresp goes to 01:

```
haddr during error = 0x400130XX  ← txd descriptor address
                  or 0x400XXXXX  ← txbuf/rxbuf address

Compare to your printed addresses:
txbuf[0] = 0x40013080  (example)
txbuf[1] = 0x40013100
txbuf[2] = 0x40013180  ← if haddr = this when error = frame 2 is problem
txbuf[3] = 0x40013200
```

Look at hwrite to know TX or RX error:

```
hwrite = 0 during error → TX DMA error (reading txbuf)
hwrite = 1 during error → RX DMA error (writing to rxbuf)
```

Step 4 — Check the AHB Arbiter

Also add the arbiter signals to see if it is a grant timeout:

```
testbench
  └── leon3soc
        └── ahbctrl0 (or ahbctrl)
              └── hmaster    ← which master currently owns the bus
                  hgrant     ← grant signals for each master
```

What a grant timeout looks like:

```
hbusreq  ──┐                                ┌──
           └────────────────────────────────┘

           GRETH waiting for bus...

hgrant   ──────────────────────────────────   ← NEVER granted!

htrans   ──00──────────────────────────────   ← GRETH stuck in IDLE
```

If GRETH never gets `hgrant` it will timeout and report an AHB error.

Step 5 — Check the APB Side Too

While the AHB error is happening on the master side, also look at:

```
testbench
  └── leon3soc
        └── apbctrl
              └── greth APB slave signals:
                    psel    ← APB select (your code polling STATUS)
                    paddr   ← which register being read
                    prdata  ← value returned

Look for:
  paddr = 0x04 (STATUS register offset)
  prdata changing — this is your poll loop reading STATUS
  When prdata bit5 (TXAHB) or bit4 (RXAHB) goes to 1 → error occurred
```

Step 6 — Timeline to Look For

Zoom into the waveform around the time of frame 2 (third TX_EN pulse):

```
Time →

TX_EN   ──┐       ┌──┐       ┌──┐       ┌──
          └───────┘  └───────┘  └───────┘

          frame0      frame1      frame2
                                  ↑
                            Look here for AHB error

AHB signals during frame2 TX:
  haddr   = txbuf[2] address
  hwrite  = 0 (reading)
  hresp   = 01? ← ERROR

APB STATUS register:
  prdata  = 0x20? ← bit5 (TXAHB) set
```

Quick Checklist for DVE

```
□ Add ahbmo.haddr    — see which address caused error
□ Add ahbmo.hwrite   — 0=TX DMA read, 1=RX DMA write
□ Add ahbmo.htrans   — see when GRETH initiates transfers
□ Add ahbmi.hresp    — look for 01 (ERROR) ← main signal to find
□ Add ahbmi.hready   — look for 0 (wait states before error)
□ Add ahbmi.hgrant   — confirm GRETH gets bus access
□ Add GRETH_STATUS   — see bit5(TXAHB) or bit4(RXAHB) set in register
□ Zoom to frame2/3 TX_EN pulse area
□ Find exact simulation time when hresp = 01
□ Check haddr at that time — which buffer address failed
□ Check hwrite — TX or RX DMA that failed
```

Once you find the time when `hresp = 01` and the `haddr` value at that moment, you will know exactly which buffer address GRETH could not access — and that tells you precisely what the fix needs to be.

can u give me a document with all the signals i need to check for all the tests with their expected outputs